

```

import cv2 # Import OpenCV for image processing algorithms
import numpy as np # Import Numpy for mathematical matrix operations
import matplotlib.pyplot as plt # Import Matplotlib to visualize the output on screen

# =====
# STEP 1: CREATE SYNTHETIC FACE TARGET (Image Acquisition Model)
# =====
# Generate a simple 120x120 raw scene frame containing sensor noise
raw_capture = np.ones((120, 120), dtype=np.uint8) * 40
cv2.circle(raw_capture, (60, 60), 40, 200, -1) # Simulated Head
cv2.circle(raw_capture, (45, 50), 6, 60, -1) # Left Eye
cv2.circle(raw_capture, (75, 50), 6, 60, -1) # Right Eye
cv2.ellipse(raw_capture, (60, 80), (15, 8), 0, 0, 180, 60, 3) # Mouth

# Simulate high-frequency noise interference from sensor capture
noise = np.random.normal(0, 15, raw_capture.shape).astype(np.int16)
acquired_feed = np.clip(raw_capture.astype(np.int16) + noise, 0, 255).astype(np.uint8)

# =====
# STEP 2: LOW-LEVEL CV (Preprocessing & Image Enhancement)
# =====
# Smooth out grain variance while maintaining large structural boundaries
low_level_clean = cv2.GaussianBlur(acquired_feed, (5, 5), 0)

# =====
# STEP 3: MID-LEVEL CV (Face Localization / Segmentation)
# =====
# Threshold to isolate the foreground structure from background pixels
_, thresh = cv2.threshold(low_level_clean, 100, 255, cv2.THRESH_BINARY)
contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

face_crop = low_level_clean.copy()
box_x, box_y, box_w, box_h = 0, 0, 0, 0

for c in contours:
    if cv2.contourArea(c) > 1000: # Identify valid facial structure area
        box_x, box_y, box_w, box_h = cv2.boundingRect(c)
        # Crop out structural region matrix bound exclusively to the face target
        face_crop = low_level_clean[box_y:box_y+box_h, box_x:box_x+box_w]

# =====
# STEP 4: HIGH-LEVEL CV (Feature Interpretation & Decision Logic)
# =====
# Quantify simple spatial structure metrics (e.g., Aspect Ratio of the face)
face_width = float(box_w)
face_height = float(box_h)
aspect_ratio = face_width / face_height if face_height > 0 else 0

# Semantic Decision Engine Rule
# (In production systems, this uses a deep embedding vector distance comparison)
if 0.8 <= aspect_ratio <= 1.2:
    identity_decision = "MATCH: TARGET AUTHORIZED"
else:
    identity_decision = "ALERT: UNKNOWN TARGET"

# Render high-level bounding metrics context box onto output display
decision_feed = cv2.cvtColor(acquired_feed.copy(), cv2.COLOR_GRAY2RGB)
if box_w > 0:
    cv2.rectangle(decision_feed, (box_x, box_y), (box_x + box_w, box_y + box_h), (0, 255, 0), 2)

# =====
# STEP 5: VISUALIZATION (Compact layout display matching pipeline stages)
# =====
fig, axes = plt.subplots(1, 3, figsize=(7, 2.5))

axes[0].imshow(acquired_feed, cmap='gray')
axes[0].set_title('Low-Level:\nAcquired Feed + Noise', fontsize=7)
axes[0].axis('off')

axes[1].imshow(face_crop, cmap='gray')
axes[1].set_title('Mid-Level:\nSegmented Face Crop', fontsize=7)
axes[1].axis('off')

axes[2].imshow(decision_feed)
axes[2].set_title(f'High-Level Context:\n{identity_decision}', fontsize=7, color='green' if "MATCH" in identity_decision else 'red')
axes[2].axis('off')

plt.tight_layout()
plt.show()

```

Low-Level:
Acquired Feed + Noise



Mid-Level:
Segmented Face Crop



High-Level Context:
MATCH: TARGET AUTHORIZED

